

Q1: When you publish a message or call a service, why does only your turtlebot (Color) respond and not any other?

Because of the `ROS_DOMAIN_ID`. In our lab, each team's laptop and robot are set to a **unique ID number**. ROS 2 nodes can only communicate if they share the same ID. So even though everyone is publishing to the same topic name `/cmd_vel`, my commands only reach my own robot.

Q2: Is using the laser scanner a reliable strategy to avoid collisions?

Mostly reliable, but not perfect. It works well for detecting walls and objects at the **same height** directly. However, the laser only scans a single flat 2D plane, so it completely misses things like cables on the floor or an overhanging desk corner above the sensor height.

Q3: What is the detection range of the laser scanner?

For the LDS-01 on our TurtleBot3 Burger (from ROBOTIS documentation):

- Minimum: 0.12 m — anything closer is a blind spot and returns `0.0` or `inf`
- Maximum: 3.5 m
- Full 360° scan

In our code, we only use two narrow windows: *front* (index 0–15 and 345–360, i.e. **±15°**) and *rear* (index 165–195, i.e. **±15° around 180°**), since we only care about what is directly **ahead** or **behind** the robot.

Q4: How is the index of the laser scan data mapped to the turtlebot coordinates?

ROS uses the convention where **forward** is the **positive X-axis** and **left** is the **positive Y-axis**. The laser array starts from the front and counts counter-clockwise:

- `ranges[0]` → front (0°)
- `ranges[90]` → left (90°)
- `ranges[180]` → rear (180°)
- `ranges[270]` → right (270°)

Q5: What are the other components of the `/cmd_vel` message good for?

The `Twist` message has 6 fields: `linear.x/y/z` and `angular.x/y/z`. For our TurtleBot3, only `linear.x` (forward/backward) and `angular.z` (turning) actually working.

The rest are designed for other platforms —

`linear.y` for sideways sliding (omnidirectional robots),

`linear.z` for up/down movement (drones),

`angular.x/y` for 3D tilting. On our robot, these are simply ignored by the firmware.

Q6: Explain `BEST_EFFORT` and `KEEP_LAST` in the context of QoS.

- **BEST_EFFORT** : Send the data and **don't bother retrying** if a packet is lost over Wi-Fi. This is fine for laser scans because we only care about the **latest frame**, not ones from a second ago.
- **KEEP_LAST with depth=1** : Only keep the **single newest message** in the queue. When a new scan arrives, the old one is thrown away immediately.

Q7: Visualize the timing of your program.

	t = 0 s	t = 0 ~ 1.5 s	t = 1.5 ~ 3.0 s	t = 3.0 s+
State	WALK	BACKING_UP	TURNING	WALK
LaserScan	Clear	Obstacle detected	Not checked	Clear
/cmd_vel	Random fwd/back	Reverse 0.08 m/s	Spin 0.5 rad/s	New random speed
Robot action	Moving randomly	Backs away	Spins to new direction	Moving randomly again

Timer (every 2 s): picks a new random speed only when `state == WALK` . `scan_callback` runs at ~5 Hz and drives all state transitions.

The scan callback runs every time a new laser frame arrives. In WALK state it checks for obstacles and updates the command. In BACKING_UP and TURNING, it just keeps sending the escape command and counts elapsed time until the phase is done.

Q8: What are the weaknesses of your randwalk_safe_turtle.py solution?

1. **Side blind spots**: We only check **$\pm 15^\circ$ at the front and rear**. When the robot is turning, it has no idea if there is a wall right beside it and could easily scrape its side against an obstacle.
2. **No obstacle check during escape**: In the BACKING_UP and TURNING states, we **stop reading the laser** completely. If there is a wall behind the robot while it is reversing, it will drive straight into it.
3. **Fixed safety threshold**: The 0.2 m trigger distance does **not adapt to the robot's current speed**. At higher speeds the robot needs more braking distance, but our code treats every situation the same way.